

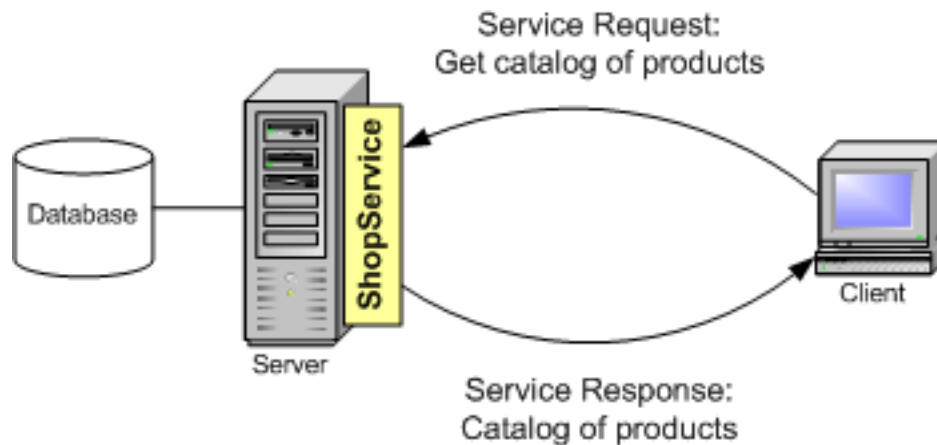
บทที่ 6

กริดเซอร์วิส

6.1 เว็บเซอร์วิส

เป็นเทคโนโลยีการคำนวณแบบกระจายที่สร้างแอปพลิเคชันในไคลเอนต์/เซิร์ฟเวอร์ (client/server) เช่น สมมติว่า เราต้องพัฒนาแอปพลิเคชันสำหรับโชคลงสินค้าที่มีอยู่ทั่วประเทศแต่ราย ชื่อสินค้าอยู่ในฐานข้อมูลที่สำนักงานกลางเท่านั้น ดังนั้นซอฟต์แวร์ที่คลงสินค้าเหล่านั้นต้องสามารถเข้าถึงรายชื้อสินค้าที่สำนักงานกลางได้ ทำได้โดยผ่านเว็บเซอร์วิสที่ชื่อ ShopService (ไม่ควรให้ใช้บนเว็บไซต์ทั่วไป เพราะทุกคนสามารถเห็นสิ่งเหล่านั้นได้ ซึ่งต่างจากการใช้เว็บเซอร์วิสเพราะเป็นการเรียกใช้ผ่านซอฟต์แวร์)

ไคลเอนต์ที่ต้องการใช้เว็บเซอร์วิสจะส่งการร้องขอไปยังเซิร์ฟเวอร์ เพื่อขอรายการต่างๆจากนั้น เซิร์ฟเวอร์ก็จะส่งรายการนั้นมาให้เป็นการตอบรับของเซอร์วิส ดังภาพด้านล่าง



รูปที่ 6-1 การตอบรับของเซอร์วิส

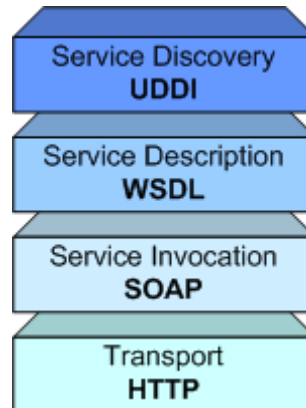
ข้อดีของเว็บเซอร์วิส ประการแรกคือ ไม่ขึ้นกับแพลตฟอร์มและไม่ขึ้นกับภาษาที่ใช้เขียน เนื่องจากใช้ภาษามาตรฐาน XML ซึ่งหมายความว่าเว็บเซอร์วิสที่เขียนด้วยภาษา JAVA บน Linux Server สามารถทำงานบนเครื่อง Windows ที่มีโปรแกรมภาษา C++ ประการที่สองคือ เว็บเซอร์วิส เกือบทั้งหมดใช้ HTTP ในการรับ-ส่งข้อมูล (เช่น การร้องขอ และ ตอบกลับของเซอร์วิส) เป็นผลดีต่อการทำแอปพลิเคชันที่ใช้งานบนอินเทอร์เน็ต เนื่องจาก HTTP ไม่มีปัญหากับพร็อกซีและไฟร์วอลล์

6.1.1 โครงสร้างของเว็บเซอร์วิส

จากรูปที่ 6-2 แสดงให้เห็นโครงสร้างของเว็บเซอร์วิสที่มีองค์ประกอบดังนี้

- Service Discovery : ช่วยให้สามารถทราบเว็บเซอร์วิสที่เหมาะสมกับความต้องการ โดยใช้ UDDI

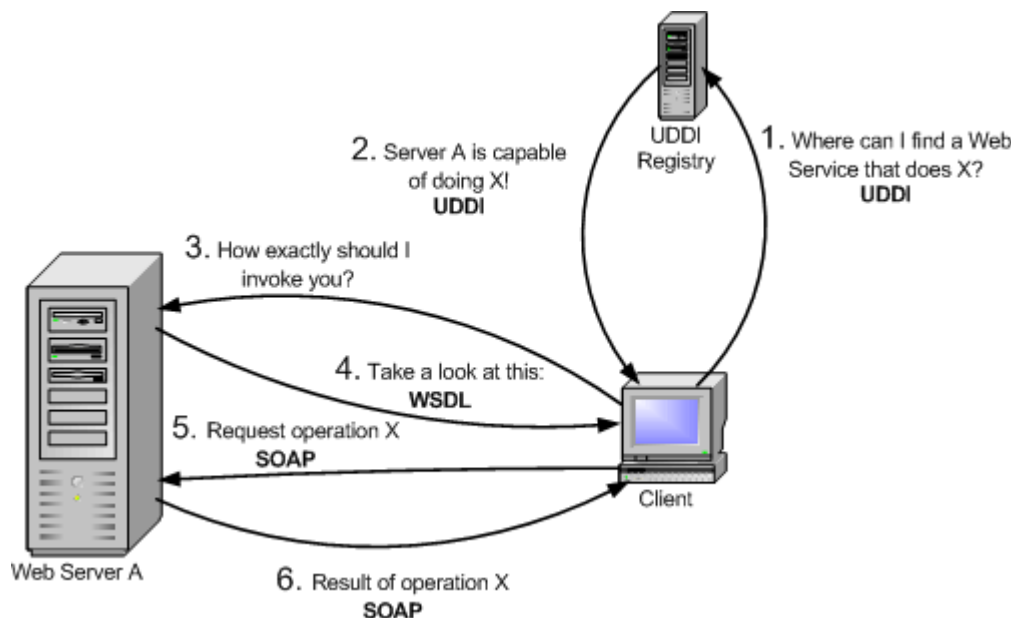
- Service Description : เมื่อรู้ว่าจะติดต่อ เว็บเซอร์วิส ได้ที่ไหนแล้ว เราก็สามารถขอให้ เว็บเซอร์วิส นั้นอธิบายอธิบายตัวเองในแง่ของแอปพลิเคชันที่ใช้, คำสั่งที่มี, วิธีเรียกใช้ โดย WSDL



รูปที่ 6-2 โครงสร้างของเว็บเซอร์วิส

- Service Invocation : การเรียกใช้ เว็บเซอร์วิส รวมไปถึงการส่งข้อความระหว่างไคลเอนต์และเซิร์ฟเวอร์มักจะใช้ SOAP ในการกำหนดรูปแบบที่ใช้
- Transport : ใช้ HTTP ในการรับ-ส่ง ระหว่างไคลเอนต์กับเซิร์ฟเวอร์

6.1.2 การเรียกใช้ เว็บเซอร์วิส



รูปที่ 6-3 แสดงลำดับการเรียกใช้เว็บเซอร์วิส

จากรูปที่ 6-3 แสดงให้เห็นขั้นตอนในการเรียกใช้เว็บเซอร์วิส

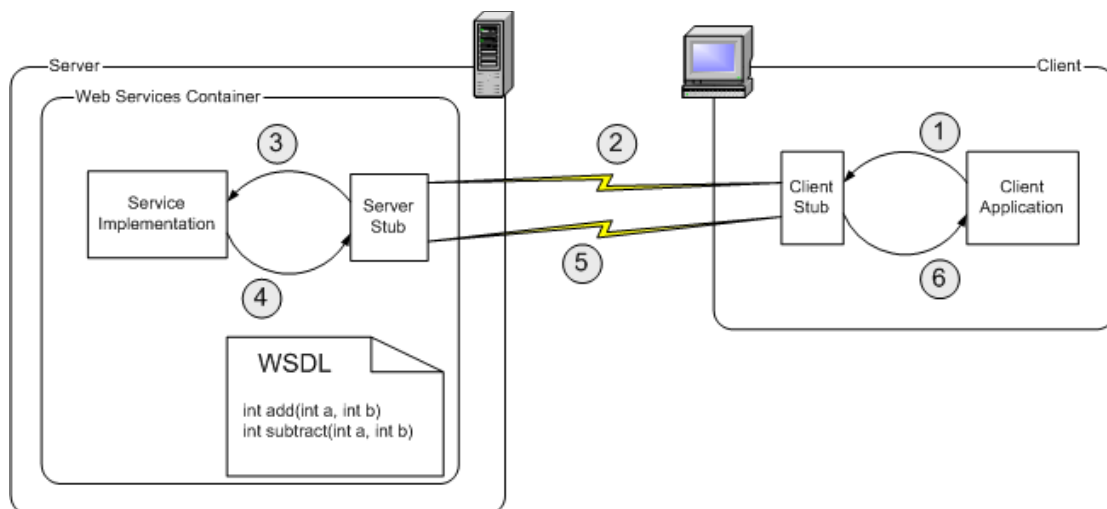
1. ไคลเอนท์ไม่จำเป็นต้องมีความรู้เกี่ยวกับเว็บเซอร์วิสที่ตัวเองกำลังจะเรียกใช้มาก่อนโดยจะเริ่ม ค้นจากค้นหาเว็บเซอร์วิสที่เหมาะสมกับความต้องการของเราโดยติดต่อไปยัง UDDI registry
2. UDDI จะตอบกลับว่ามีเซิร์ฟเวอร์ไหนมีเซอร์วิสที่เราต้องการ
3. เมื่อรู้เว็บเซอร์วิสแล้วแต่ก็ยังไม่ว่าจะเรียกใช้อย่างไรต้องถามไปยังเว็บเซอร์วิส นั้นให้อธิบายวิธีเรียกใช้
4. เว็บเซอร์วิสจะส่ง WSDL มาให้
5. เมื่อรู้เว็บเซอร์วิสและวิธีเรียกใช้ก็จะเกิดการเรียกใช้ขึ้นเองโดยภาษา SOAP ส่งการร้องขอไปยัง เว็บเซอร์วิส
6. เว็บเซอร์วิส ตอบกลับมามี SOAP response พร้อมสิ่งที่เราต้องการ หรืออาจจะเป็นข้อความแสดง ความผิดพลาด ถ้า SOAP ที่ส่งไปนั้นผิด

6.1.3 แอปพลิเคชันของเว็บเซอร์วิส

เมื่อไคลเอนท์ต้องการเรียกใช้เว็บเซอร์วิสเราสามารถสร้างทาสก์ (task) แทรกไว้ใน ซอฟต์แวร์ เรียกว่า client stub สำหรับเรียกใช้เว็บเซอร์วิสแทนที่จะเขียน SOAP หรือ WSDL ที่ละบรรทัด ซึ่งหลายๆ เครื่องมือในปัจจุบันมีการสร้าง client stub ให้อย่างอัตโนมัติ โดยยึดเอา WSDL เป็นพื้นฐาน ดังนั้นขั้นตอน ที่ถูกต้องในการเรียกใช้เว็บเซอร์วิส เป็นดังนี้

1. เรารู้ที่ตั้งของเว็บเซอร์วิสที่เหมาะสมกับความต้องการผ่าน UDDI
2. เราได้รับ WSDL ของ เว็บเซอร์วิส นั้น
3. เราสร้าง stub ขึ้นมาแทรกไว้ในแอปพลิเคชัน
4. แอปพลิเคชันจะใช้ stub นั้นในแต่ละครั้งที่ต้องการเรียกใช้เว็บเซอร์วิส

เราสามารถสร้างฟังก์ชันทั้งหมดของ เว็บเซอร์วิส ขึ้นมา แล้วสร้าง server stub จาก WSDL เพื่อใช้ แปลการร้องขอและส่งต่อให้เซอร์วิสที่สร้างขึ้นมา และเมื่อเซอร์วิสนั้นได้ผลลัพธ์ก็จะส่งไปให้ server stub ซึ่งจะทำการสร้าง SOAP response ที่เหมาะสม ในการจัดการกับเซอร์วิสและ server stub จะใช้ ผ่าน ซอฟต์แวร์ที่เรียกว่า เว็บเซอร์วิสคอนเทนเนอร์ (Web Service container) เพื่อให้แน่ใจว่า HTTP request ที่จะ เข้ามายัง เว็บเซอร์วิส ได้เข้ามาที่ server stub ได้อย่างถูกต้อง



รูปที่ 6-4 ลำดับการเรียกใช้เว็บเซอร์วิส

สมมติว่า เราารู้ที่ตั้ง เว็บเซอร์วิส แล้ว และสร้าง client stubs จาก WSDL และ โปรแกรมทางฝั่ง เซิร์ฟเวอร์ก็สร้าง server stubs แล้ว สามารถอธิบายขั้นตอนการเรียกใช้ เว็บเซอร์วิส ได้ดังนี้

1. เมื่อไคลเอนต์ต้องการเรียกใช้เว็บเซอร์วิสก็จะทำการเรียก client stub ซึ่งจะทำให้การแปลงคำขอให้ เป็น SOAP request เรียกขั้นตอนนี้ว่า marshaling หรือ serializing process
2. SOAP request ถูกส่งผ่านเครือข่ายโดยใช้ผ่านโปรโตคอล HTTP จากนั้นเว็บเซอร์วิสคอนเทนเนอร์ ก็จะได้รับ SOAP request และส่งให้ server stub ซึ่งทำการเปลี่ยน SOAP request ให้อยู่ในรูปแบบที่ เซอร์วิสสามารถเข้าใจได้ เรียกขั้นตอนนี้ว่า unmarshaling หรือ deserializing
3. เซอร์วิสได้รับการร้องขอ จาก server stub และทำงานตามการร้องขอนั้น
4. ผลลัพธ์ที่ได้ถูกส่งไปยัง server stub และถูกแปลงเป็น SOAP response
5. SOAP response ถูกส่งผ่านเครือข่ายผ่านโปรโตคอล HTTP เมื่อ client stub ได้รับก็จะทำการแปลง SOAP response นั้นให้อยู่ในรูปแบบที่แอปพลิเคชันเข้าใจได้
6. แอปพลิเคชันได้รับผลลัพธ์ที่ต้องการจากเว็บเซอร์วิสไปใช้

6.2 กริดเซอร์วิส

กริดเซอร์วิส เป็นส่วนขยายของเว็บเซอร์วิสที่ขยายขึ้นตามมาตรฐานที่กำหนดไว้ใน Open Grid Service Infrastructure เพื่อนำมาใช้งานในการสร้างระบบกริด ในหัวข้อต่อไปนี้จะกล่าวถึงมาตรฐานที่ใช้ กำหนดกริดเซอร์วิส ให้ทราบก่อน

6.2.1 OGSA (Open Grid Service Architecture)

เป็นสิ่งที่กำหนดโดย The Global Grid Forum เพื่อใช้กำหนดมาตรฐานแอปพลิเคชันของกริดและเพื่อให้สามารถใช้งานได้จริง เนื่องจากกริดเซอร์วิสเป็นเทคโนโลยีการคำนวณแบบกระจายภายใต้ OGSA จึงกล่าวได้ว่า OGSA เป็นรากฐานของกริดเซอร์วิส

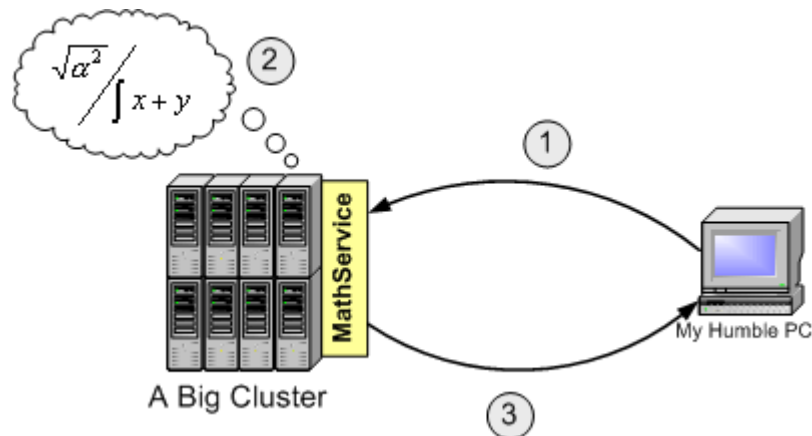
6.2.2 OGSi (Open Grid Service Infrastructure)

เนื่องจาก OGSA เป็นเพียงโครงร่างของ กริดเซอร์วิส แต่ไม่ได้อธิบายรายละเอียดถึงขั้นตอนการใช้งานจริง จึงได้พัฒนาให้มี OGSi ที่ใช้ระบุวิธีการทำงานของ กริดเซอร์วิส โดยละเอียด

6.2.3 คุณสมบัติหลักที่ได้รับการพัฒนาใน OGSi

- Stateful and potentially transient services

เป็นลักษณะสำคัญอย่างหนึ่งที่เกี่ยวข้องกับเว็บเซอร์วิสยกตัวอย่างเพื่อความเข้าใจดังนี้ สมมติถ้าเรามีกลุ่มคลัสเตอร์หรือคอมพิวเตอร์ ขนาดใหญ่ที่สามารถประมวลผลงานต่างๆ ได้ ตั้งอยู่ที่สำนักงานในกรุงเทพ และต้องการให้พนักงานที่เชียงใหม่, ขอนแก่น และชลบุรี สามารถใช้งาน คลัสเตอร์นี้ได้อย่างสะดวก โดยที่เราอาจจะสร้าง เว็บเซอร์วิสที่ใช้คำนวณทางคณิตศาสตร์ขึ้นมา เรียกว่า MathService ซึ่งมีคำสั่ง SolveReallyBigSystem(), SolveFermatsLastTeorem()



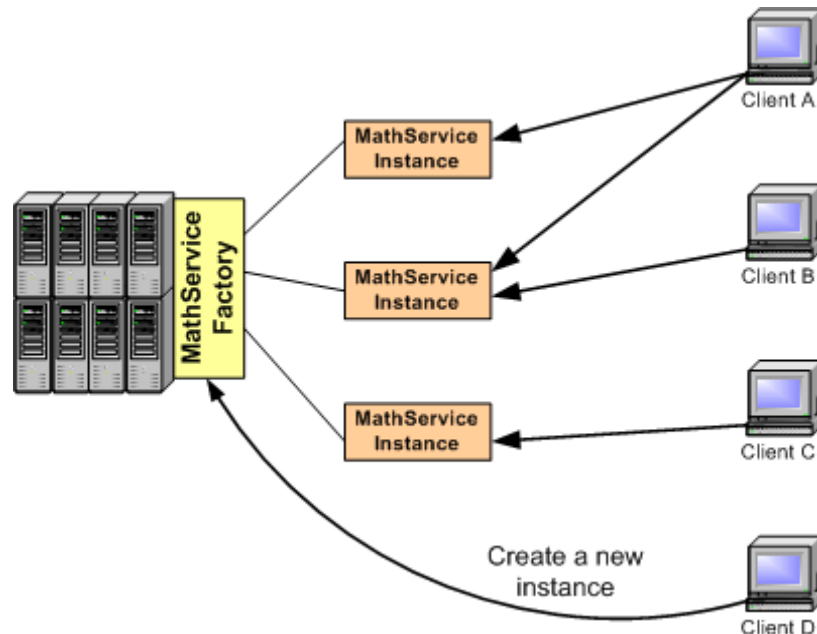
รูปที่ 6-5 ลำดับการทำงานแบบมีสเตตของ OGSi

และอื่นๆ ให้ใช้งานการทำงานของเซอร์วิสจะเป็นตามขั้นตอนดังนี้

1. เรียกใช้ MathService โดยขอใช้คำสั่งต่างๆ ของเซอร์วิสนั้น
2. MathService จะสั่งให้กลุ่มคลัสเตอร์ทำงานตามคำสั่งที่ขอไว้
3. MathService จะคืนผลลัพธ์ที่ได้กลับมาให้

อย่างไรก็ตาม หากว่ากันตามจริงแล้ว เราอาจต้องการให้คลัสเตอร์ทำงานให้เราหลายคำสั่ง ซึ่งมีความเกี่ยวข้องกันในคราวเดียว ถ้าเป็นเว็บเซอร์วิสซึ่งเป็น stateless แล้วล่ะก็ จะไม่สามารถจดจำได้ว่าการเรียกใช้แต่ละคำสั่งได้ ว่ามีความสัมพันธ์กันอย่างไร แต่ในกริดเซอร์วิสนั้น จะแก้จุดนั้นโดย

ใช้แฟคทอรี (factory) ที่เมื่อไคลเอนต์ต้องการสร้างหรือลบอินสแตนซ์ (instance) ต้องบอกผ่านแฟคทอรีอินสแตนซ์ มีอายุการใช้งานชั่วคราวเท่าที่ไคลเอนต์ต้องการเพื่อจะได้ไม่ต้องขึ้นกับอายุการใช้งานของกริดเซอร์วิสคอนเทนเนอร์แต่จะขึ้นอยู่กับแอปพลิเคชันแต่ใช้ว่าทุก กริดเซอร์วิสจะต้องใช้แฟคทอรี หรือ อินสแตนซ์เท่านั้น แต่กริดเซอร์วิสสามารถรอให้บริการ



รูปที่ 6-6 แสดงการเรียกใช้ MathService Instance

ตลอดเวลาเหมือนเว็บเซอร์วิสทั่วไปได้ตามแต่สมควรกับแอปพลิเคชันนั้นๆ

- Lifecycle management

การจัดการกับวัฏจักรชีวิตของเซอร์วิสโดยตรงนั้นเป็นเรื่องยาก (ถ้าใช้ผ่านแฟคทอรีหรืออินสแตนซ์เนื่องจากอินสแตนซ์จะถูกสร้างและลบทิ้งเมื่อไหร่ก็ได้)จึงต้องมี lifecycle management มาช่วยจัดการในเกี่ยวกับวัฏจักรชีวิตของกริดเซอร์วิส

- Service Data

Service Data มีความเป็น stateful และใช้งานได้แบบชั่วคราว เป็นสิ่งสำคัญที่ กริดเซอร์วิส มีเพิ่มจาก เว็บเซอร์วิส เพื่อช่วยให้เราสามารถเพิ่มโครงสร้างข้อมูล เข้าไปในทุกๆเซอร์วิส ทำให้สามารถเข้าถึงคำสั่งต่างๆได้โดยตรงผ่านอินเตอร์เฟซของมันเอง โดยข้อมูลที่เพิ่มเข้าไปมักเป็น

- State information : ให้ข้อมูลสแตตปัจจุบันของเซอร์วิส
- Service metadata : ข้อมูลเกี่ยวกับตัวเซอร์วิสเอง

- Notifications

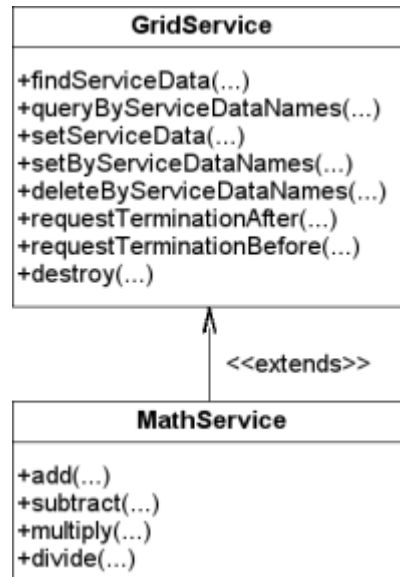
สามารถใช้ กริดเซอร์วิส เป็นแหล่งประกาศ (notification) และไคลเอนท์เป็นผู้รับประกาศนั้นเมื่อมีการเปลี่ยนแปลงบางอย่างเกิดขึ้นกับกริดเซอร์วิสจะมีการแจ้งไปยังไคลเอนท์ทุกตัว (แจ้งเฉพาะการเปลี่ยนแปลงบางอย่างตามแต่ผู้เขียนโปรแกรมของกริดเซอร์วิสต้องการ)

- Service Groups

เราสามารถนำเซอร์วิส ต่างๆมาจับกลุ่มเข้าไว้ด้วยกัน แล้วเรียกใช้ได้ครั้งเดียว โดยสามารถเพิ่มหรือลบเซอร์วิสออกจากกลุ่มได้

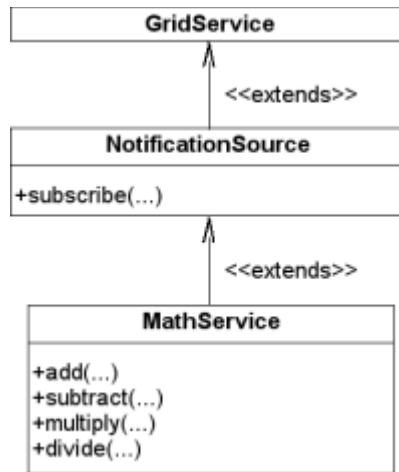
- portType Extension

เรียกอินเตอร์เฟซของ เว็บเซอร์วิส ว่า portType ซึ่งโดยทั่วไปมักจะมีเพียง 1 portType ต่อ 1 เว็บเซอร์วิส แต่ใน กริดเซอร์วิส สามารถมีได้หลายๆ portType ต่อ 1 เซอร์วิส ซึ่ง portType ที่เพิ่มขึ้นมา นั้นต้องขยายมาจาก portType ที่มีอยู่ก่อน



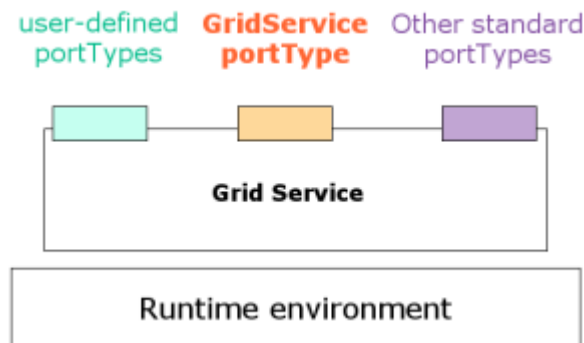
รูปที่ 6-7 แสดงการขยาย portType ของกริดเซอร์วิส

OGSI ออกมาตรฐาน portType อื่นๆที่สามารถขยายได้โดยการเพิ่มฟังก์ชัน เข้าไปใน กริดเซอร์วิส



รูปที่ 6-8 แสดงการเพิ่มฟังก์ชันเข้าไปในกริดเซอร์วิส

โดยทั่วไปแล้ว จะพบว่า กริดเซอร์วิส มี portType อยู่ 3 อย่าง



รูปที่ 6-9 แสดง portType ของกริดเซอร์วิส

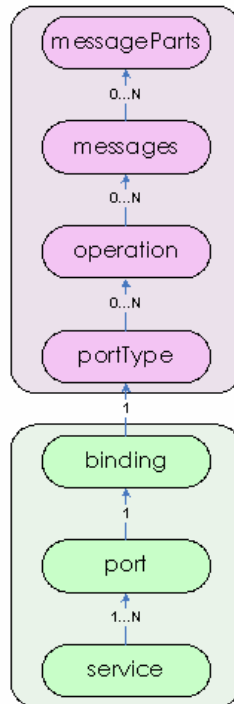
- GSH & GSR

กริดเซอร์วิส ถูกระบุตำแหน่งด้วย URI เช่นเดียวกับ เว็บเซอร์วิส แต่เรียก กริดเซอร์วิส URI ว่า Grid Service Handle (GSH) เช่นเดียวกับ เว็บเซอร์วิส ที่ต้องมี WSDL ใช้อธิบายวิธีติดต่อกับ เว็บเซอร์วิส นั้นๆ กริดเซอร์วิส ก็ต้องมี Grid Service Reference (GSR)

6.3 ความแตกต่างของกริดเซอร์วิสและเว็บเซอร์วิส

6.3.1 Web Services Description Language (WSDL)

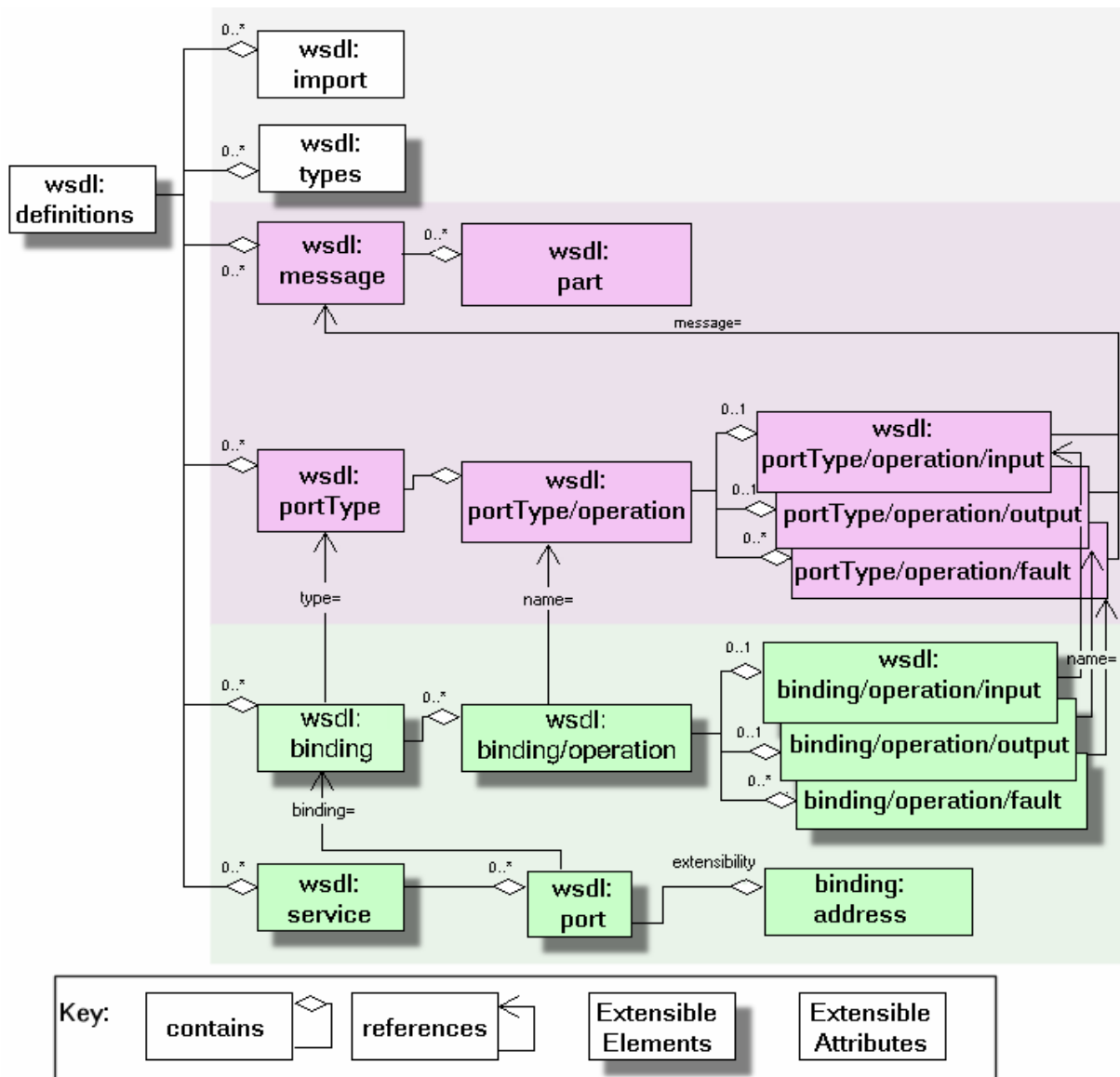
WSDL เป็น XML document เพื่ออธิบายข้อตกลงระหว่างเว็บเซอร์วิสและผู้บริโภคเกี่ยวกับ รูปแบบของข้อความที่สามารถเปลี่ยนเป็นรูปแบบใดก็ได้บ้างที่เซอร์วิสจะรองรับ จากรูป 6-10 เป็นโครงร่าง และความสัมพันธ์ของโครงสร้างในส่วนหลักของ WSDL document



รูปที่ 6-10 โครงสร้างเอกสาร WSDL

องค์ประกอบของ document ถูกแบ่งออกเป็น 2 ส่วน:

- ส่วนบน ('message', 'portType') เป็นส่วนสำคัญของ service interface, operations, parameters ซึ่งหมายความว่าผู้บริโภครสามารถทำอะไรได้บ้าง
- ส่วนล่าง ระบุ binding(s) ของวิธีการทำให้ message formats เป็นรูปธรรมขึ้นมา, โปรโตคอล และที่อยู่ปลายทางที่สามารถเรียกใช้เซอร์วิสได้



รูปที่ 6-11 ค่าต่างๆของ WSDL

จากรูป 6-11 อธิบายโครงสร้างของ WSDL document ละเอียดขึ้น เริ่มจาก ส่วนประกอบ XML ที่เรียกว่า wsdl:definitions อาจประกอบด้วยส่วนย่อยๆ เช่น wsdl:import และ wsdl:types

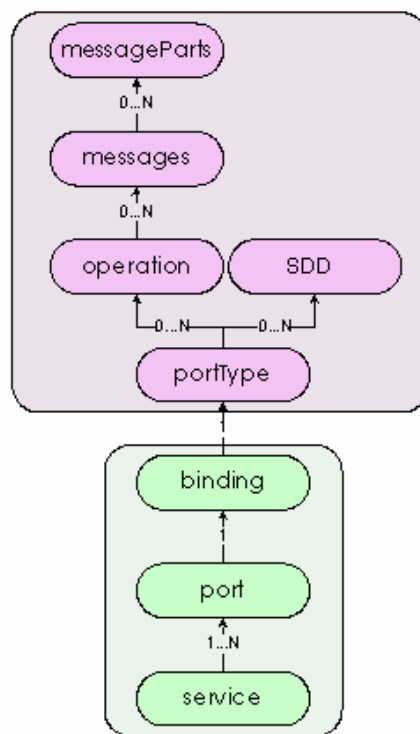
หัวใจของ WSDL คือ portType ซึ่งระบุกลุ่ม input, output และ fault message ที่เซอร์วิส เตรียมรับมือไว้แล้ว บางครั้งก็เทียบง่ายๆได้ว่า portType เป็น JAVA interface หรือ C++ class ส่วน message ใน portType อาจประกอบด้วยหลายๆส่วนซึ่งแต่ละส่วนอาจต่างชนิดกัน หรืออาจเป็น input และ output parameters ที่เมสเสจรวมไว้ด้วยกัน

ชนิดของเมสเสจถูกระบุด้วยชนิดของส่วนประกอบใน wsdl:definition ซึ่งสามารถขยายออกไปได้โดยการเข้าไปสร้างส่วนประกอบย่อยเพื่อสร้างชนิดของข้อมูล โดยชนิดโดยปกติใน WSDL คือ XML Schema

ส่วนสำคัญอื่นๆของ WSDL คือการนำมาผูกกัน (binding) เป็นการสร้างเมสเสจซึ่งก็คือ data encoding, messaging protocol, และ underlying communication protocol ส่วนประกอบ XML ที่ใช้จะเป็น operations ที่ใช้ชื่ออ้างอิงถึงเมสเสจต่างๆ ลักษณะสำคัญของ WSDL คือความจุที่สามารถขยายได้โดยการเพิ่มส่วนประกอบย่อยๆเข้าไป WSDL ยอมให้บรรยาย data encoding, message และ communication protocol ใดๆได้ ประโยชน์ของการบรรยายส่วนต่างๆขึ้นกับระบบไคลเอนท์และ เซิร์ฟเวอร์สามารถแปล WSDL descriptions เพื่อเข้าและถอดรหัสเมสเสจ

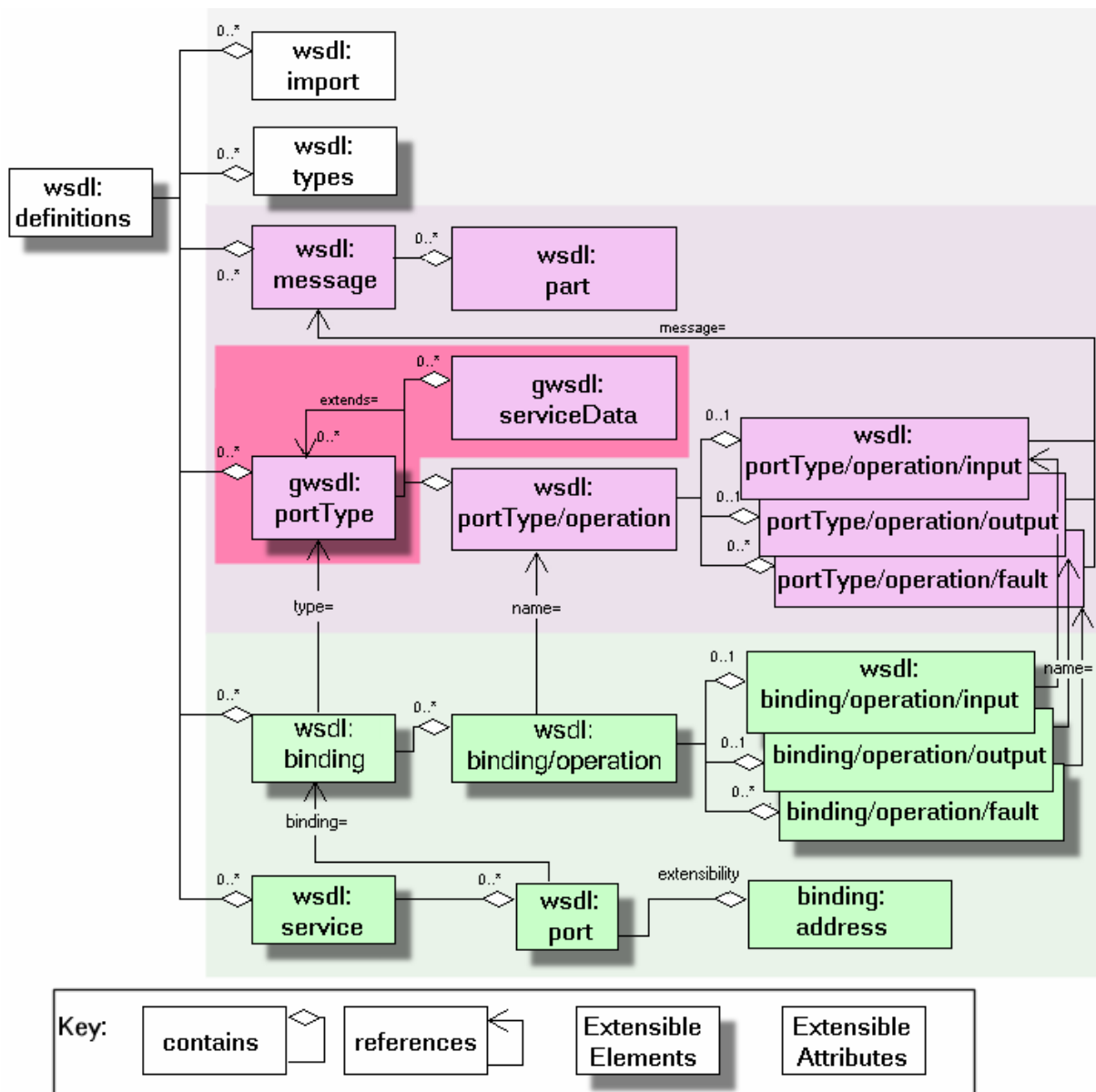
6.3.2 Grid Services Description Language (GWSDL)

OGSI เพิ่ม features ของกริดเซอร์วิสเข้าไปในเว็บเซอร์วิสโดยทำ WSDL portType element ซ้ำ โดยกริดเซอร์วิส จะบรรยายในรูปแบบของ WSDL ที่เพิ่มขึ้น ที่เรียกว่า GWSDL รูป 6-12 แสดงโครงสร้างหลักของ GWSDL ซึ่งคล้ายกับโครงสร้างของ WSDL



รูปที่ 6-12 โครงสร้างเอกสาร GWSDL

ข้อแตกต่างระหว่าง GWSDL กับ WSDL แสดงอยู่ในรูปต่อไปนี้



รูปที่ 6-13 Markup language for Grid Services

- ใน OGSI, portType หนึ่งๆ ถูกสร้างโดยการอ้างอิงถึง portType หรือ portTypes ที่มีอยู่กับ attribute ที่เพิ่มขึ้น และการกำหนดความหมายเพิ่มเข้าไปเท่าที่ต้องการ
- ขณะที่ WSDL portType มีแค่ operations, OGSI portType ก็มี Service Data Declarations (SDDs) ซึ่งช่วยบรรยาย identity และ interfaces ของ service รวมไปถึงสถานะที่ไคลเอนท์ สามารถดูได้

6.3.3 ตัวอย่างโปรแกรม 'Counter'

จะเปรียบเทียบให้เห็นความแตกต่างระหว่างกริดเซอร์วิส และ เว็บเซอร์วิส โดยอาศัยโปรแกรม Counter ช่วยในการเปรียบเทียบ

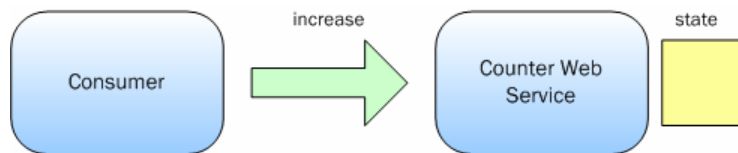
6.3.3.1 Counter Web Service

WSDL สำหรับ Counter Service ทัวไปที่มี 2 operations (increase และ getValue) แสดงดังรูป 6-14

```
<wsdl:definitions xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:counter="http://www.gridforum.org/namespaces/
2003/05/ogsiprimer/counter/webservice"
  targetNamespace="http://www.gridforum.org/namespaces/
2003/05/ogsiprimer/counter/webservice">
  <wsdl:types>
    <xs:schema/>
  </wsdl:types>
  <wsdl:message name="increaseMsg">
    <wsdl:part name="value" type="xs:positiveInteger"/>
  </wsdl:message>
  <wsdl:message name="getValueMsg">
    <wsdl:part name="value" type="xs:positiveInteger"/>
  </wsdl:message>
  <wsdl:portType name="counterPortType">
    <wsdl:operation name="increase">
      <wsdl:input message="increaseMsg"/>
    </wsdl:operation>
    <wsdl:operation name="getValue">
      <wsdl:output message="getValueMsg"/>
    </wsdl:operation>
  </wsdl:portType>
</wsdl:definitions>
```

รูปที่ 6-14 เอกสาร WSDL Counter Web Service

ในโลกของ Web Services, Counter Web Service คือ agent ที่คอยแสดงผลเป็น WSDL และเป็นตัวรับ operation requests



รูปที่ 6-15 การใช้คำสั่งเพิ่มค่าใน Counter Web Service

ถ้าทำ Counter Web Service ให้เป็น multiple counter ต้องมี id เพิ่มเข้ามาเพื่อใช้ระบุ counter ที่กำลังถูกใช้งาน ตามที่แสดงในรูป 6-16



รูปที่ 6-16 การใช้คำสั่งเพิ่มค่าใน Counter Web Service ที่รองรับหลายๆ Counter

```

<wsdl:definitions xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:counter="http://www.gridforum.org/namespaces/
2003/05/ogsiprimer/counter/basic "
  targetNamespace="http://www.gridforum.org/namespaces/
2003/05/ogsiprimer/counter/basic ">
  <wsdl:types>
    <xs:schema/>
  </wsdl:types>
  <wsdl:message name="increaseMsg">
    <wsdl:part name="counterId" type="xs:positiveInteger"/>
    <wsdl:part name="value" type="xs:positiveInteger"/>
  </wsdl:message>
  <wsdl:message name="getValueRequestMsg">
    <wsdl:part name="counterId" type="xs:positiveInteger"/>
  
```

```

</wsdl:message>
<wsdl:message name="getValueResponseMsg">
  <wsdl:part name="value" type="xs:positiveInteger"/>
</wsdl:message>
<wsdl:portType name="counterPortType">
  <wsdl:operation name="increase">
    <wsdl:input message="increaseMsg"/>
  </wsdl:operation>
  <wsdl:operation name="getValue">
    <wsdl:input message="getValueRequestMsg"/>
    <wsdl:output message="getValueResponseMsg"/>
  </wsdl:operation>
</wsdl:portType>
</wsdl:definitions>

```

รูปที่ 6-17 เวอร์ชันแก้ไขของ Counter Web Services WSDL ที่รองรับหลายๆ counters

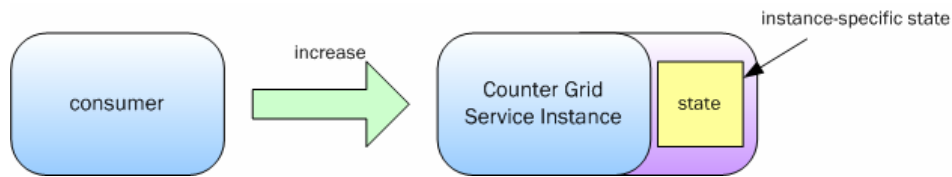
6.3.3.2 Counter Grid Service

ใน OGSI-terms, กริดเซอร์วิส ใช้อ้างอิงถึงแม่แบบอย่างเป็นทางการ เป็นข้อตกลงที่ว่า interface ที่แสดงเป็น GWSDL ต้องมีกริดเซอร์วิสอินสแตนซ์ติดมาด้วย แสดงถึงลักษณะส่วนประกอบของแอปพลิเคชันของกริด (ไม่นับรวมถึง agent ที่สามารถรับ operation request ได้) หมายถึง กริดเซอร์วิสอินสแตนซ์ เป็นตัวรับ logical ของ operation requests

ในมาตรฐานของ OGSI, จำเป็นต้องมี กริดเซอร์วิสอินสแตนซ์ ที่กำหนด Grid Service Handle (GSH) ซึ่งถูกระบุขึ้นมาเป็นพิเศษ และมีการลงทะเบียน แล้วจึงให้ กริดเซอร์วิสอินสแตนซ์ อื่นๆหาเจอได้ ขณะที่ GSH มีลักษณะพิเศษ เพื่อสร้าง Grid Service Reference (GSR) ขึ้นมาเพื่อให้ได้ endpoint-related information ซึ่งจำเป็นต้องใช้ในการเข้าถึงกริดเซอร์วิสอินสแตนซ์ที่ระบุไว้

ครั้งหนึ่ง ผู้ใช้สามารถใช้งาน GSR เพื่อทำให้กริดเซอร์วิสอินสแตนซ์ทำงานได้เหมือนกับ Web Service counterpart อย่างไรก็ตาม ลักษณะโดยเฉพาะของ OGSI ของกริดเซอร์วิสอินสแตนซ์นั้นสามารถใช้ประโยชน์ได้มากกว่านี้

Counter GSI (Counter Grid Service Instance) ดังแสดงในรูป 3-18 เหมือนกับ Counter Web Service ที่ได้แสดงไปแล้วข้างต้น ผู้ใช้งานเซอว์วิสหลายคนสามารถขอคำสั่งบน Counter GSI เดียวกันได้ เพื่อเข้าถึงหรือแก้ไขสถานะของ counter นี่เป็นข้อแตกต่างอย่างหนึ่งจาก Web Service ที่ต้องติดต่อเป็น logic ไปยัง อินสแตนซ์



รูปที่ 6-18 การใช้คำสั่งเพิ่มค่า Counter Grid Service Instance

ด้วยความแตกต่างจาก Counter Web Service ทำให้ Counter Grid Service Instance ต้องมีคุณสมบัติในการจัดการโลไฟท์ม์ ซึ่งใช้จัดการกับวัฏจักรชีวิตของ counter

GWSDL interface ของ Counter Grid Service คล้ายกับ เว็บเซอว์วิส นอกจากเปลี่ยน namespace อย่างเห็นได้ชัดแล้ว ข้อแตกต่างอื่นๆดังรูปด้านล่าง แสดงถึง GWSDL processors ที่ counterPortType ขยายการใช้งานออกไปด้วย ogsi:GridService portType

```

<wsdl:definitions xmlns:ogsi="http://www.gridforum.org/namespaces/2003/03/OGSI"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:counter="http://www.gridforum.org/namespaces/
2003/05/ogsiprimer/counter/basic"
  targetNamespace="http://www.gridforum.org/namespaces/
2003/05/ogsiprimer/counter/basic">
  <wsdl:types>
    <xs:schema/>
  </wsdl:types>
  <wsdl:message name="increaseMsg">
    <wsdl:part name="value" type="xs:positiveInteger"/>
  </wsdl:message>
  <wsdl:message name="getValueMsg">
    <wsdl:part name="value" type="xs:positiveInteger"/>
  </wsdl:message>

```



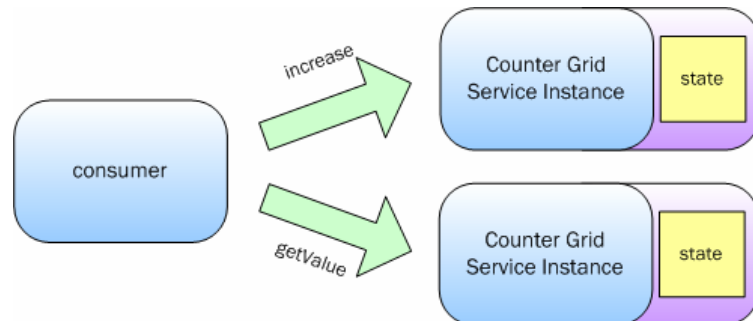
```

</wsdl:message>
<ogsi:portType name="counterPortType" extends="ogsi:GridService">
  <wsdl:operation name="increase">
    <wsdl:input message="increaseMsg"/>
  </wsdl:operation>
  <wsdl:operation name="getValue">
    <wsdl:output message="getValueMsg"/>
  </wsdl:operation>
</wsdl:portType>
</wsdl:definitions>

```

รูปที่ 6-19 เอกสาร GWSDL ของ Counter Grid Service

แน่นอนว่าความสัมพันธ์ระหว่างกริดเซอร์วิสอินสแตนซ์และสเตทของมันยอมให้เราดำเนินการใช้งานของ Multiple-Counter Web Service โดยไม่ส่งผลกระทบต่อ interface ทั้งหมดนี้ กล่าวคือ กริดเซอร์วิสอินสแตนซ์เป็นรูปแบบใหม่จาก Web Service ดังรูป 6-18 เวลาต้องการใช้งาน counter หลายครั้ง ค่าเดิมจะยังคงอยู่ใน interface เดิม อย่างไรก็ตาม OGSI ก็สนับสนุนให้ กริดเซอร์วิสอินสแตนซ์ แสดงสเตทได้ชั่วคราว ดังนั้น จึงเป็นธรรมดาที่จะแสดงแต่ละ counter ด้วยแต่ละ กริดเซอร์วิสอินสแตนซ์ กล่าวคือ มีความสัมพันธ์แบบ 1 ต่อ 1 ระหว่างทรัพยากรหนึ่ง(ในที่นี้คือ counter) กับกริดเซอร์วิสอินสแตนซ์



รูปที่ 6-20 การใช้ 2 Counter GSI ที่แตกต่างกันที่ใช้ในแต่ละ counter

นอกจากการจัดการสเตทแล้ว กริดเซอร์วิสอินสแตนซ์ สามารถเข้าถึงข้อมูลผ่านโครงสร้างที่เรียกว่า Service Data Elements (SDEs) ที่ประกาศใน interface ของกริดเซอร์วิสผ่านทาง คำสั่งที่ระบุโดย GridService portType ของ OGSI

Counter Grid Service อาจเผยค่าของ counter ผ่าน SDE โดยไม่ต้องพึ่งคำสั่ง getValue รูป 3-21
แสดง GWSDL ของ Counter Grid Service ที่มี SDE

```
<wsdl:definitions xmlns:ogsi="http://www.gridforum.org/namespaces/2003/03/OGSI"
  xmlns:sd="http://www.gridforum.org/namespaces/
2003/03/serviceData"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:counter="http://www.gridforum.org/namespaces/
2003/05/ogsiprimer/counter/basic"
  targetNamespace="http://www.gridforum.org/namespaces/
2003/05/ogsiprimer/counter/basic">
  <wsdl:types>
    <xs:schema/>
  </wsdl:types>
  <wsdl:message name="increaseMsg">
    <wsdl:part name="value" type="xs:positiveInteger"/>
  </wsdl:message>
  <wsdl:message name="getValueMsg">
    <wsdl:part name="value" type="xs:positiveInteger"/>
  </wsdl:message>
  <ogsi:portType name="counterPortType" extends="ogsi:GridService">
    <wsdl:operation name="increase">
      <wsdl:input message="increaseMsg"/>
    </wsdl:operation>
    <sd:serviceData name="counterValue"
      type="xs:positiveInteger"
      minOccurs="1"
      maxOccurs="1"
      mutability="mutable">
```

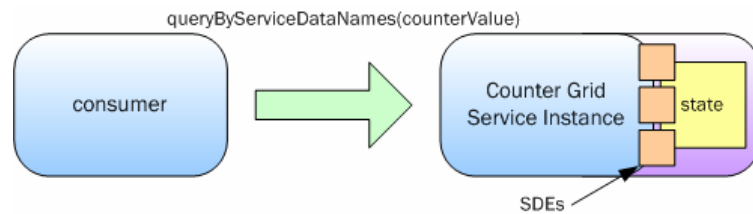
```

<sd:documentation>
    The value of the counter.
</sd:documentation>
</sd:serviceData>
</ogsi:portType>
</wsdl:definitions>

```

รูปที่ 6-21 GWSDL สำหรับ Counter Grid Service ที่ใช้ค่า SDE

ตอนนี้จึงเป็นไปได้ที่จะเข้าถึง counterValue SDE ผ่าน operations ที่กำหนดโดย GridService portType (รูป 6-22)



รูปที่ 6-22 การร้องขอค่า counter จากลูกค้าผ่าน counterValue SDE